

The Code Blitz: *ninety minutes, the whole application*

You took two weeks over Lab 1. Today you have ninety minutes, and you are going to ship something several times its size — because you will be driving an agentic IDE rather than typing.

ONBOARDING	BUILD TIME	FEATURE TARGET	TOTAL MARKS
45 min	90 min	50 pts	70

Demonstration: on your own machine, during the session. You show; the demonstrator marks what they see.

2.1 Brief

You are going to build the todo application from Lab 1 again.

Same stack: **Next.js** and **SQLite**, local-first, no accounts, a single user on the machine it runs on. Same rules as Lab 1 — a task cannot be deleted, only archived; the three statuses are fixed; overdue is derived, never stored; everything survives a restart.

Start from an **empty project**. Do not open your Lab 1 work. The first twenty minutes of this lab rebuild all of Lab 1 from nothing, and that is the point: the work that took you a fortnight is now a checkpoint you clear before the real exercise begins.

2.2 How the ninety minutes run

CLOCK	CHECKPOINT	BEFORE YOU MOVE ON
0–20	Core. The whole Lab 1 application, running: every item in the Core area of the feature menu.	It runs, and you can create and archive a task.
20–90	Blitz. Work the feature menu, one feature at a time. Finish and check each one before starting the next.	The application still runs and the tests still pass.

THE HARD RULE

At the buzzer you demonstrate whatever is running. There is no time to fix things during the demonstration. Integrate continuously. Do not have four agent tasks in flight at minute 88.

2.3 The feature menu

The **Core** area is required and is marked separately. The other six areas are in no particular order and of no particular difficulty — pick whatever combination interests you. Nothing is gained by spreading across areas or by staying inside one.

Beyond Core, claim features totalling **at least 50 points**. Points beyond 50 earn no extra marks, but they are useful insurance: if a feature is judged partial or rejected at the demonstration, the overflow covers it.

Each entry states what *done* means. That sentence is the bar:

- **Full points** — the *Done when* condition is met, in front of the demonstrator.
- **Half points** — the feature is there and does something real, but the *Done when* condition is not met.
- **Nothing** — absent, broken, or not reachable from the UI (or, for the interop area, from a documented command).

Anything claimed must also survive a restart of the application.

CORE – §2.5 *required, marked separately — this is Lab 1, and it earns no feature points*

CODE	PTS	FEATURE
CA1	REQ	Create a task carrying Title, Description, Due Date and Topic, which then appears in the list. DONE WHEN all four fields are captured and visible afterwards.
CA2	REQ	Edit a task , so that the change outlives the process. DONE WHEN you edit a task, stop the application, start it again, and the change is still there.
CA3	REQ	Archive a task , never delete one. It leaves the active list but remains viewable. DONE WHEN an archived task is still findable, and nothing in the application removes a row.
CA4	REQ	Sort the list by topic, by status and by due date. DONE WHEN all three orderings work and are correct, not merely offered.
CA5	REQ	Flag overdue tasks , derived from the due date rather than stored, and never as a fourth status. The three statuses are Todo, In-Progress and Complete, and they are fixed. DONE WHEN a past-due task is visibly marked and <i>overdue</i> does not appear in the status selector.

DATA AND SCHEMA

CODE	PTS	FEATURE
DS1	6	Subtasks. A task can contain child tasks, to any depth. The parent shows aggregate progress derived from its children rather than a separately stored figure, and archiving a parent handles its children coherently — whichever rule you choose, be ready to say why. DONE WHEN a three-level tree exists, ticking a leaf moves the root’s progress, and the tree survives a restart.

CODE	PTS	FEATURE
DS2	5	Tags. Free-form labels, many per task and many tasks per label, stored as a proper join rather than a comma-separated string. Includes creating, renaming, colouring and deleting a tag, and filtering by several at once. DONE WHEN renaming a tag updates every task carrying it, and a two-tag filter returns the intersection.
DS3	4	Projects. A grouping one level above topic — a task belongs to at most one project — with a per-project view showing its tasks and completion. DONE WHEN a project page lists only its own tasks and reports a progress figure that changes as they complete.
DS4	8	Dependencies. A task can be blocked by one or more others. The UI shows what is blocking a task and what it blocks, a task is not startable while blocked, and an attempt to create a cycle is refused with a message naming the offending path. DONE WHEN $A \rightarrow B \rightarrow C \rightarrow A$ is rejected on the attempt, not after the fact, and completing B visibly unblocks C.
DS5	8	Recurrence. Daily, weekly, monthly and a custom interval. Completing an instance closes it and creates the next at the correct date; the series can be edited or ended without destroying past instances. DONE WHEN a monthly task due on the 31st produces a sensible next date in February, and completing an instance leaves the completed one visible in history.
DS6	3	Priority and effort. A fixed priority scale (P0–P3) and an effort estimate, both filterable and sortable, and both surfaced in the list without opening a task. DONE WHEN sorting by priority then due date gives a stable, correct order.

INPUT AND SEARCH

CODE	PTS	FEATURE
IS1	8	Natural-language quick add. One text box that parses a sentence into a full task: <code>submit report next friday 5pm #coms3011 !p1 every week</code> sets title, due date, topic, priority and recurrence. A live preview shows the parsed result before it is saved, and unparsed text stays in the title rather than being silently dropped. DONE WHEN three different phrasings of a relative date all resolve correctly, and the preview updates as you type.
IS2	7	Filter query language. A search box accepting a real query syntax — <code>is:overdue topic:algorithms due:<2026-08-20 tag:reading</code> — with a parser, support for negation and at least one boolean operator, and an error message that points at the offending token rather than failing silently. DONE WHEN a malformed query produces a useful error and no results are silently wrong.

CODE	PTS	FEATURE
IS3	5	Full-text search. Search over title and description backed by SQLite FTS5 rather than a <code>LIKE</code> scan, returning ranked results with the matching phrase highlighted in a snippet, and staying fast against the seed data. DONE WHEN a mid-word or multi-word phrase returns highlighted snippets, and the index is kept current when a task is edited.
IS4	5	Command palette. <code>Ctrl/Cmd-K</code> opens a single input that fuzzy-matches both commands (“archive task”, “switch to calendar”) and tasks by title, executing on Enter, fully keyboard-driven. DONE WHEN an action and a specific task are both reachable in under five keystrokes without touching the mouse.
IS5	4	Keyboard-only operation. Every action in the application — create, edit, status change, archive, navigate views — reachable without a mouse, with visible focus at all times and a shortcut overlay on <code>?</code> . DONE WHEN the entire Core walkthrough can be performed with the mouse untouched.

VIEWS AND INTERACTION

CODE	PTS	FEATURE
VW1	7	Kanban board. The three statuses as columns, tasks as cards, drag between columns to change status. The change persists on drop, the card renders its key fields, and the board respects the active filter. DONE WHEN a dragged card is in its new column after a reload, and a failed write reverts the card rather than lying.
VW2	7	Calendar. Month and week views placing tasks on their due dates, with overdue and completed visually distinct, navigation between periods, and drag-to-reschedule that writes the new due date. DONE WHEN dragging a task to another day changes its due date everywhere in the application, and a day with many tasks degrades gracefully.
VW3	4	Today view. A single focused screen for someone about to start work: overdue first, then due today, then anything in progress — ordered deliberately, and you can say why that order. DONE WHEN the view is empty-stated properly when there is nothing to do, rather than showing a blank panel.
VW4	6	Timeline. Tasks laid out horizontally against time, with bars spanning start to due date and arrows drawn between dependent tasks, scrollable and zoomable across at least a month. DONE WHEN a dependency chain of three tasks is legible as a chain, not as three unrelated bars.

CODE	PTS	FEATURE
VW5	4	Saved views. A filter, sort and view-type combination the user can name, save, return to and delete, persisted in the database rather than in browser storage. DONE WHEN a saved view is still there after a restart and reproduces exactly the list it was saved from.
VW6	5	Scale. The list stays responsive at 10,000 tasks — virtualised rendering, indexed queries, pagination or windowing at the database level, and no full-table load into memory. DONE WHEN the seeded 10,000-row database scrolls smoothly and the list's first paint is not visibly slower than at 50 rows.

TIME AND HISTORY

CODE	PTS	FEATURE
TH1	6	Activity log. An append-only table recording every mutation — what changed, when, and the values before and after — written in the same transaction as the change itself, never edited or deleted, and viewable as a reverse-chronological feed. DONE WHEN every kind of mutation appears in the feed, and archiving a task adds a row rather than removing any.
TH2	7	Undo and redo. <code>Ctrl-Z</code> and <code>Ctrl-Shift-Z</code> across at least create, edit, archive and status change, reconstructed from the activity log rather than from in-memory state, with a redo stack that clears correctly on a new action. DONE WHEN five mixed operations can be undone in order and redone back to where you started.
TH3	4	Task history. Per task, a readable account of what changed and when — field, old value, new value, timestamp — rather than a raw event dump. DONE WHEN a task edited three times shows three legible entries, in order, naming the fields that changed.
TH4	5	Statistics. Completion over time as a chart, a per-day activity heatmap, current and longest streak, and a breakdown by topic — all computed from real data, with the empty case handled. DONE WHEN the figures move correctly after completing a task, and a fresh database renders without crashing.
TH5	4	Start dates and snooze. A start date distinct from the due date, so a task can exist without yet being actionable, plus a snooze action that pushes the start date forward. Not-yet-started tasks are hidden from Today but never from search. DONE WHEN a task starting tomorrow is absent from Today and present in the full list, and snoozing moves it.

INTEROP AND PLATFORM

CODE	PTS	FEATURE
IP1	5	Import and export. JSON, CSV and Markdown, both directions, round-trip safe — exporting a database and importing it into an empty one reproduces it exactly, including archived tasks and relationships. Import reports what it rejected rather than failing halfway. DONE WHEN an export-then-import cycle produces an identical row count in every table, and a malformed row is reported by line.
IP2	4	iCalendar export. A <code>.ics</code> file that a real calendar client can open or subscribe to, with correct field mapping and escaping, and a documented URL or file path to hand to the client. DONE WHEN the file opens in an actual calendar application with the right titles on the right dates.
IP3	5	Backup, restore and migrations. A one-command backup and restore, plus versioned schema migrations that upgrade a database created by an earlier build of your own application instead of dropping and recreating it. DONE WHEN a database made before your last few changes still opens under the current build with its data intact.
IP4	7	REST API with an OpenAPI 3 specification. Full CRUD over HTTP with correct status codes and error bodies, described by a specification file that matches the implemented routes, served as a browsable documentation page. DONE WHEN a request made from the docs page against the running application succeeds, and the spec has no route the code lacks.
IP5	5	CLI client. <code>npm run todo add "..."</code>, <code>list</code>, <code>done</code> and similar, driving the same core the web UI uses — one implementation of the logic, not two — with output readable in a terminal and a non-zero exit code on failure. DONE WHEN a task added from the CLI appears in the running web UI, and the shared code path is identifiable.
IP6	8	MCP server. Expose your todo database as tools over the Model Context Protocol, so that an agent inside Qoder can list, create and complete your tasks. Requires a tool schema, a documented way to connect it, and sane behaviour when the agent sends nonsense. DONE WHEN you drive your own application by asking the agent, live, in front of the demonstrator.

QUALITY AND CONFIDENCE

CODE	PTS	FEATURE
QC1	8	A test suite of twenty or more real tests. Behaviour, not renders. Must include the awkward cases: recurrence across a month boundary, the timezone at which a task becomes overdue, dependency cycle rejection, and archive semantics. Deterministic, and run against a throwaway database from a single documented command. DONE WHEN the suite passes twice in a row and no test depends on the contents of your working database.

CODE	PTS	FEATURE
QC2	6	End-to-end test. A Playwright run that drives a real browser through the entire Core walkthrough — create, edit, restart, archive, sort, overdue — and fails loudly when any step breaks. DONE WHEN it runs from one documented command and you can show it going red by breaking a feature.
QC3	3	Seed script. Generates several hundred plausible tasks with realistic titles, spread of due dates, topics, statuses and relationships — enough to make the views and the scale work look like something. DONE WHEN one command produces a populated database and the data does not look like Task 1, Task 2, Task 3 .
QC4	5	Accessibility pass. Keyboard focus managed across dialogs and views, labels and roles on interactive elements, colour not the only carrier of meaning, and a passing automated axe check wired into the test command. DONE WHEN the axe check runs from the test command and reports no violations on the main views.
QC5	3	Appearance settings. Dark mode and a density setting, honouring the system preference by default, overridable by the user, and persisted across a restart. DONE WHEN the chosen theme survives a restart and neither mode has unreadable text anywhere.

2.4 Working with the agent

This lab is a conversation, not a dictation. The agent is excellent at execution and poor at guessing your intent, so most of your ninety minutes is spent describing, reading what came back, and saying what was wrong with it. Some of this you will have seen in the onboarding session; it is repeated here because it is what separates a working application at the buzzer from a broken one.

- **Say the constraint before it is violated, not after.** The rules carried over from Lab 1 — overdue derived, archive as a flag, three fixed statuses, nothing ever deleted — are exactly the ones an agent will break unprompted. State them at the start of a task, and restate them whenever you open a new one.
- **Ask for one thing, then look.** A task you can check in thirty seconds is the right size. “Build the whole application” is not a task, it is an abdication.
- **Push back in specifics.** “That’s wrong” gets you a different guess. “You stored `overdue` as a column; derive it from `due_date` at read time instead” gets you the fix. The quality of what you get out is set by the precision of what you object to.
- **Slice into independent pieces.** Tags, the calendar view and the export format do not touch each other. Features that all rewrite the task mutation path do. Run independent work concurrently; run the rest in sequence.
- **Run the tests after every feature.** The failure mode of a blitz is silent breakage four features back.
- **Read the output.** You are responsible for every line in the project, including the ones you did not type.

MISTAKES AGENTS GENUINELY MAKE ON THIS BRIEF – ALL OF WHICH COST YOU MARKS AT THE DEMONSTRATION

- ✗ `overdue` stored as a column, or added as a fourth status. The Core area forbids both.
- ✗ Archiving implemented as a `DELETE`, or by copying the row to another table.
- ✗ Recurrence that produces 31 February, or drifts by an hour across a DST boundary.
- ✗ Overdue computed against the server's timezone rather than the user's, so tasks flip at the wrong midnight.
- ✗ A list view that issues one query per task to fetch its tags.
- ✗ No index on the columns every query filters by.
- ✗ Tests that mock the database and therefore assert only that the mocks were called.
- ✗ A dependency graph walk with no cycle guard, which hangs the process on the first loop.
- ✗ Migrations that drop and recreate tables, destroying existing data.
- ✗ Three different libraries installed to do one job, because they were added by three separate tasks.

2.5 Marking

Total: 70 marks — Core 20, features 50. Everything is marked live, at your machine, from what you show.

CORE – 20 MARKS

The five Core features, 4 marks each, pass or fail with no partial credit. Cosmetic defects are not penalised. If the application does not start, the whole lab scores zero.

FEATURES – 50 MARKS

One point equals one mark, capped at 50:

VERDICT	AWARD
Yes — meets its <i>Done when</i> condition	full points
Partial — present and doing something real, but the <i>Done when</i> condition is not met	half points
No — absent, broken, or unreachable	nothing